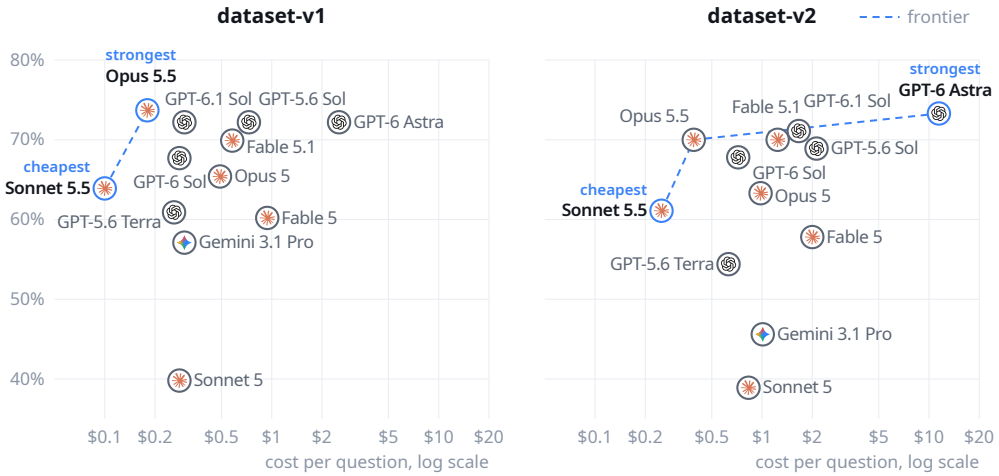
What Production Reveals That Leaderboards Can't

Lingjie Kong
Energent.ai
Stanford, CA 94305
kong@energent.ai

Rachel Hu
Energent.ai
Stanford, CA 94305
rachel@energent.ai

Abstract

We trusted the leaderboards and shipped their top model to production. But our users reported poor performance, and the leaderboards couldn't say why. So we ran twelve frontier models from OpenAI, Anthropic, and Google DeepMind on OfficeQA Pro and OfficeQA Pro V2, benchmarks of data-analysis tasks over real U.S. government financial reports. All twelve models share the same harness, which scores near or above published results despite lower reasoning effort, so the harness is not the bottleneck. With everything else fixed, we find that a single leaderboard score hides three things a production team needs. **(i) Grader:** switching from the benchmark's old grading rules to its new ones drops scores by up to 28 points on the same answers. **(ii) Strategy:** models answer the same questions in very different ways. Two models near the top differ: GPT-6 Astra hands whole documents to subagents, while Claude Fable 5.1 searches for only what it needs. Gemini 3.1 Pro mostly runs code, rarely opens the documents, and scores far below both. **(iii) Cost:** Astra costs 9× more per question than Fable 5.1, for a 3-point lead within single-run noise. The 9× comes from 3.9× more tokens and a 2.3× higher price per token. A score is only as honest as the work behind it. We will release all of ours, every step of every run.



Dataset dataset-v1 → dataset-v2 · 2× corpus (697 → 1,435) · 3.7× docs/question (1.8 → 6.7) · 10× text/question (0.97 → 9.3 MB)
Harness our model-agnostic data-analysis harness; reasoning effort high for every model

Figure 1: Twelve frontier models on the two OfficeQA datasets, dataset-v1 and dataset-v2, accuracy against cost per question; only the model and its endpoint differ between points.

Preprint.

1 Introduction

In February 2026, a new frontier model launched at the top of several public leaderboards. We switched our production data-analysis agent to it, and our users soon reported poor performance on their knowledge-work tasks, which resemble OfficeQA's. The leaderboard could not tell us why. This paper is about what it leaves out.

A published score is one number, and it depends on who produced it. For example, on Humanity's Last Exam, Anthropic reports Claude Fable 5.1 at 60.9 without tools and 65.0 with tools [1]; Scale AI's board reports the same model at 46.5, below both [2]. One model, one benchmark, three numbers from two producers. Worse, the answers are graded by a language model, and each producer uses a different one: Claude Opus 4.6 for Anthropic, o3-mini for Scale [1, 2]. Such judges are known to disagree with one another (Section 2). And none of the three numbers shows how the model solved the tasks or what its cost was made of.

So we ran the models ourselves. We use OfficeQA [3, 4], a public benchmark of data-analysis tasks over real U.S. government financial reports, each needing retrieval, reasoning, and code (Section 2). We run twelve frontier models from OpenAI, Anthropic, and Google DeepMind on both of its datasets, OfficeQA Pro, which we call dataset-v1, and the harder OfficeQA Pro V2, which we call dataset-v2, in one model-agnostic harness with everything fixed except the model and its endpoint (Section 3), and we keep every trajectory. OfficeQA's grader is code, not a language model, so we can re-score the same answers exactly under two of its versions.

Figure 1 shows what a leaderboard would print from these runs: one accuracy and one cost per model. We look at three things a team serving an agent in production needs but a leaderboard leaves out: how the grader measured the score (Section 5.1), what strategy the agent used to solve each task (Section 5.2), and what drives the cost of each request (Section 5.3). We ask teams running agents in production to monitor all three, not just accuracy and cost.

2 Related Work

Inputs. A benchmark score has inputs and outputs. The inputs are the settings that produced it, including who ran it: the harness, prompt, reasoning effort, corpus, and grader, among others. Each can move a score on its own. On Chatbot Arena, providers that test private variants and disclose only the best bias the ranking [5]. Language-model judges disagree with humans and with each other and favor their own outputs [6, 7]. Reasoning effort and token budget are test-time compute, and test-time compute moves accuracy by margins larger than the gaps between frontier models; forcing a longer or shorter reasoning budget alone shifts a competition-math score by seven points [8–10]. Sampling temperature, seeds, and single runs introduce variance that typical benchmark sizes do not resolve, and reported reasoning gains often fall inside it [11–13]. Where the harness is recorded, its choice shifts agentic results by margins comparable to the model gaps and shifts cost by far more [14–17]; where the prompt is recorded, its wording alone reorders rankings [18–20]. In the terms of Raji et al. [21], a score is a property of an evaluation setup rather than of a model.

Outputs. The outputs are what the score conceals about the run: how the grader shaped it, how the model went about solving the task, and what the dollar was made of. On grading, the ABC checklist [22] audits agentic benchmarks for grader rigor and finds most under-specified, and a grader's answer-extraction and format rules change scores without changing the model [23–25]. On strategy, Cuadron et al. [10] read 4,018 agent trajectories to relate reasoning behavior to outcome, and HAL [26] logs full trajectories across agents as evaluation infrastructure. On cost, Kapoor et al. [27] argue that agent benchmarks must report cost beside accuracy, and Cost-of-Pass [28] turns the pair into a frontier. Prior work has examined each output on its own; we examine all three together, on OfficeQA.

Why OfficeQA. OfficeQA's tasks are the kind of knowledge work our users do: retrieve figures from scanned financial reports, parse tables, reason across documents, write code to compute the result, and return a single final answer in the required format [3, 4]. Because the work runs through tools, the harness matters as much as the model. Each question was written by annotators, independently re-answered by a second annotator, arbitrated by a third where the answers differed, and passed through two rounds of end-to-end quality assurance. Its grader is one public, versioned file of

deterministic code, so any change to what counts as correct is a dated commit rather than a judge's drift.

Dataset-v1: OfficeQA Pro. The OfficeQA Pro report [4] defines OfficeQA Pro as the 133 questions labeled hard in Databricks' 246-question grounded-reasoning benchmark over U.S. Treasury Bulletins [3]; it is the set on which all of Databricks' results are reported, and we call it dataset-v1. The 113 questions labeled easy complete OfficeQA Full, which we do not run. The report evaluates agents in a two-by-two design, oracle pages or the whole corpus, crossed with raw PDF or parsed text, in the providers' own harnesses. Its published numbers have since moved. On July 21, 2026, a repository commit titled "Update OfficeQA Pro figures" changed every model's value in the result chart, up to 5 points in either direction, with no note of what changed[1] [29]. In between the report and that commit, the grader file had changed what counts as a correct answer three times, in May and June.

Dataset-v2: OfficeQA Pro V2. Pro V2, which we call dataset-v2, followed in August 2026 as a generalization test on a different corpus: 90 questions over 1,435 U.S. Federal Accounts of Receipts and Expenditures documents, about 120,000 pages, drawing evidence from 6.7 source documents per question against about 2 in dataset-v1 [30]. All published numbers use one grader, "a deterministic exact-match metric with 0.0% tolerance", and every published dataset-v2 number is whole corpus, with no oracle setting.

Vendor reports. Of the three model providers, only Anthropic reports OfficeQA, and only on dataset-v1 [1, 31, 32]. The cards for Claude Sonnet 5, Opus 5, Opus 5.5, and Fable 5.1 state the setup in one sentence: models are evaluated "agentically, with documents provided as extracted text in a sandboxed environment and code-execution tools available" [1, 31, 33, 34], at max effort, averaged over five trials [31, 33], and scored "exact-match" [33]. The document format is therefore known, text; the corpus scope is stated only in the card for Claude Sonnet 5.5, as "the relevant documents preselected" [32], and the grader version in none.

3 Approach

3.1 Harness

All runs use our model-agnostic data-analysis harness, whose loop, tools, task prompt, and settings are the same for every provider; only the model changes. Its environment is an E2B sandbox [35], a headless Linux container that holds the question's documents, where every tool call executes and every result is observed. The agent has 19 tools in six groups: files (read, grep), execution (bash), documents (parse, view), web (search), subagent (agent), and task tracking (todo). A run is one trajectory with no user in the loop: the prompt asks the model to explore the documents, decide between sources, calculate, and return the answer in a fixed tag.

3.2 Configuration

Every model runs under the same configuration: reasoning effort high, temperature 1, context compacted at 200k tokens, and one run per question. What differs is the model and the endpoint that serves it, listed in Table A1 in Appendix A.

3.3 Grader

Every answer is scored by the benchmark's reward function, `reward.py`, which we call the grader, at 0% tolerance. Two versions of it appear in this paper: its first release (December 9, 2025), which we call grader-v1, and its latest release (June 13, 2026), which we call grader-v2 [29]. We report grader-v2 throughout unless we compare the two. Both read the answer inside the last `<FINAL_ANSWER>` XML tag of the trajectory, or the final message if there is no tag, and test it against the target. Table 1 lists the rules of each, and Table 2 shows answers grader-v2 rejects under the rules it adds.

[1] Whole corpus as raw PDF, provider harnesses, report Table 1 (March 2026) against the chart after July 21: Gemini 3.1 Pro Preview 18.05 to 19.6, Claude Opus 4.6 48.12 to 51.1, GPT-5.4 36.09 to 41.4, Claude Opus 4.5 39.9 to 38.4. The chart caption still dates the GPT-5.4 and Opus 4.6 results to March 9, 2026.

Table 1: Grader rules, grader-v1 against grader-v2. Grader-v2 keeps grader-v1's rule and adds seven; we call the first five, which require a single direct answer, the direct-answer rules.

Rule	grader-v1	grader-v2
The target's number appears in the answer, exactly; every number, for a list target	✓	✓
The answer is one line		✓
The answer is at most 250 characters		✓
The answer has the same count of numbers as the target		✓
The answer contains no markup tag		✓
A text target must equal the answer after normalization		✓
Currency symbols and accounting negatives are normalized		✓
Bracketed lists are parsed as lists		✓

Table 2: Answers with the target value that grader-v2 rejects, one per rule it adds; all three are from Claude Sonnet 5 (Section 5.1). Answers are truncated with "…"; ↩ marks a line break.

Target answer	Extracted predicted answer	grader-v1	grader-v2	Rule it fails
1169.41 million	\$1,169.41 million (approximately \$1.17 billion)	✓	×	same count of numbers
4.815	4.815% ↩↩ (Total assets declined from \$218,901,423 thousand…)	✓	×	one line
44,463	\$44,463 million (sum of the twelve reported 1953 monthly values…), 297 characters in full	✓	×	at most 250 characters

3.4 Measurement

Accuracy and standard error. Accuracy a_m is the share of the $|Q|$ questions that grader-v2 passes; with one run per model, its standard error is the binomial $\sqrt{a_m(1-a_m)/|Q|}$.

Trajectory records. Each trajectory records its tokens in four types: input (uncached), cache read, cache write, and output (including reasoning).

Cache hit rate. The share of all tokens sent to the model that were read from cache:

$$h_m = \frac{\sum_{q\in Q}\sum_{c\in \mathrm{calls}(q)} n_{c,\mathrm{read}}}{\sum_{q\in Q}\sum_{c\in \mathrm{calls}(q)} \left(n_{c,\mathrm{in}} + n_{c,\mathrm{read}} + n_{c,\mathrm{write}}\right)}, \tag{1}$$

where Q is the set of questions in a dataset, $\mathrm{calls}(q)$ are the API calls made to answer question q, subagent calls included, and $n_{c,t}$ is the number of tokens of type t in call c; the denominator is every token sent to the model: input, cache read, and cache write.

Cache miss rate. Each call resends the previous call's context, which should come from the cache; a token of it that does not is a cache miss, billed again as a cache write. Number the calls of each conversation $c = 1, \ldots, C$, and let $s_c = n_{c,\mathrm{in}} + n_{c,\mathrm{read}} + n_{c,\mathrm{write}}$ be the context size of call c. The cache miss rate of model m is

$$\mu_m = \frac{\sum_{q\in Q}\sum_{c=2}^{C}\left(s_{c-1} - n_{c,\mathrm{read}}\right)}{\sum_{q\in Q}\sum_{c=1}^{C} s_c}. \tag{2}$$

The rest, $1 - h_m - \mu_m$, are new tokens: context sent for the first time.

Cost. Model m's mean cost per question is

$$\bar{c}_m = \frac{1}{|Q|}\sum_{q\in Q}\sum_{c\in \mathrm{calls}(q)}\sum_{t\in\{\mathrm{in},\,\mathrm{read},\,\mathrm{write},\,\mathrm{out}\}} n_{c,t}\, p_{m,t}, \tag{3}$$

where $p_{m,t}$ is model m's list price per token of type t.

Effective price per token. Model m's effective price is $\bar{p}_m = \bar{c}_m / \bar{n}_m$, its mean cost per question over its mean tokens per question across the four types, in dollars per million tokens like list prices. Since $\bar{c} = \bar{n}\,\bar{p}$, the cost ratio of any two models A and B factors into a token ratio and a price ratio,

$$\underbrace{\frac{\bar{c}_A}{\bar{c}_B}}_{\text{cost ratio}} = \underbrace{\frac{\bar{n}_A}{\bar{n}_B}}_{\text{token ratio}} \cdot \underbrace{\frac{\bar{p}_A}{\bar{p}_B}}_{\text{price ratio}} . \tag{4}$$

4 Experiments

4.1 Dataset and setting

We run both datasets: dataset-v1, the 133 questions of OfficeQA Pro as released on March 19, 2026, and dataset-v2, the 90 questions of Pro V2 as released on August 6, 2026. Published results differ in two inputs, the scope of documents the model receives and their format (Table 3). We fix both. For scope, we give the model the oracle documents, every document the question cites, in full. This is harder than oracle pages, since the model must still find what it needs inside long documents, and easier than the whole corpus, since it does not have to search every document. For format, the documents are parsed text rather than PDF. Anthropic's cards note that "OfficeQA scores are highly sensitive to the evaluation harness" and that settings "that require the model to parse the raw PDF corpus directly yield substantially lower absolute scores for all models" [1, 33]; parsed text keeps that sensitivity, and the quality of any parser, out of the comparison. Table A2 in Appendix A gives the size of each dataset in this setting: dataset-v2 documents are shorter on average, but each dataset-v2 question cites more of them, so a dataset-v2 question gives the model about ten times as much text.

Table 3: Published results in each combination of scope and format, named by the agent that produced them, with the dataset in brackets [1, 4, 30, 33]. All entries except ours and Anthropic's were run by Databricks. The first three rows run from the least to the most material given to the model. Parsed text: the PDFs converted to text and tables before the model sees them. —: no published result.

Scope	PDF	Parsed text
Oracle pages (the answer pages only)	Claude Agent SDK, Codex, Gemini CLI (dataset-v1)	Claude Agent SDK, Codex, Gemini CLI (dataset-v1)
Oracle documents (the cited documents, in full)	—	**Our model-agnostic data-analysis harness (dataset-v1, dataset-v2)**
Whole corpus (every document in the dataset)	Claude Agent SDK, Codex, Gemini CLI (dataset-v1); Claude Code, Codex, Antigravity (dataset-v2)	Claude Agent SDK, Codex, Gemini CLI (dataset-v1); Databricks Genie (dataset-v2)
Not stated	—	Anthropic's internal harness (dataset-v1, "extracted text")

4.2 Harness validation

To check that our model-agnostic data-analysis harness does not hold models back, we compare our dataset-v1 results with the two published sources for the same questions (Table 4). Databricks reports Gemini 3.1 Pro in Gemini CLI on oracle pages, an easier setting than ours, in March 2026; its grader applied the same rules as grader-v1, so we compare under grader-v1. Anthropic reports Claude Opus 5.5, Fable 5.1, and Sonnet 5.5 in its own harness at max effort, averaged over five runs. None states the grader version, so we show ours under both graders. Our runs use high effort and a single run. Our harness therefore exceeds three of the four published numbers and falls 1.7 to 2.4 points short of the fourth, in a harder setting than Databricks' and at lower effort than Anthropic's.

Table 4: Our dataset-v1 results at high effort against the published ones, most at Anthropic's max, two levels higher (low < medium < high < xhigh < max [32]). Accuracy (%) on the 133 dataset-v1 questions. Bold: above the published number; —: not comparable. Anthropic does not state its grader version, so both of ours are shown. Gemini's effort is not stated for the CLI run; the model's default is high [4]. Published: Opsahl-Ong et al. [4]; Anthropic [1, 31, 32].

	Published			Ours: our harness, oracle documents as parsed text, **high** effort	
Model	Accuracy	Effort	Harness; documents	grader-v1	grader-v2
Databricks (same rules as grader-v1)					
Gemini 3.1 Pro	46.6	high (default)	Gemini CLI; oracle pages, parsed text	**56.4**	—
Anthropic (grader version not stated)					
Claude Opus 5.5	67.7	max	internal; "extracted text"	**72.9**	**73.7**
Claude Fable 5.1	69.0	max	internal; "extracted text"	**72.2**	**69.9**
Claude Sonnet 5.5	65.6	max	internal; "relevant documents preselected and provided as extracted text"	63.2	63.9

5 Results

5.1 Accuracy

Figure 2 scores the same trajectories under both graders. The difference reaches 27.9 points: switching from grader-v1 to grader-v2 drops Claude Sonnet 5 on dataset-v1 from 67.7% to 39.8%. It also changes which model ranks first on dataset-v1: Claude Opus 5 leads under grader-v1 at 74.4%, but falls to seventh under grader-v2, where Claude Opus 5.5 leads at 73.7%.

The difference is concentrated. On dataset-v1, three models, Claude Sonnet 5, Fable 5, and Opus 5, lose 27.9, 12.0, and 9.0 points, while the other nine move by at most 2.3; on dataset-v2, Sonnet 5 and Fable 5 again lose the most, 18.9 and 5.5 points. The cause is the one Table 2 shows for Sonnet 5: these models restate the target value with other figures, or answer over several lines or at length, and grader-v2's direct-answer rules (Table 1) score such answers wrong. On dataset-v1, 39 of Sonnet 5's 133 answers contain the target value but fail these rules: 25 the same-count rule, 8 the length rule, and 6 the one-line rule. Their successors are barely affected: Claude Fable 5.1 moves by 2.3 points on dataset-v1 and not at all on dataset-v2, Claude Opus 5.5 by 0.8 and 0, and Claude Sonnet 5.5 by 0.7 and 1.1.

Neither grader is simply stricter. Grader-v2 also normalizes currency and list formatting, which is why GPT-6 Astra gains 2.2 points on dataset-v2. And grader-v1's rule, that the target appear anywhere among the answer's numbers, credits some wrong answers: asked for 36,080, Opus 5 answered 35,532 and passed, because 36,080 appeared later in its explanation. A single printed accuracy therefore depends on which grader produced it, and the choice can reorder the top of a leaderboard. From here on we report grader-v2.

An accuracy counts every miss alike. Figure A1 in Appendix B splits each model's wrong answers under grader-v2 into four kinds. The first, a right answer in the wrong format, is the grader effect above, and it falls on the same three models. The second is near misses: every model has between 7% and 17% of questions answered within 1% of the target, which a 0% tolerance scores the same as an answer that is far off. The rest are genuinely wrong numbers, from 12% of questions for GPT-6 Astra on dataset-v2 to 41% for Gemini 3.1 Pro, and they, rather than format or precision, separate the models at the bottom of the ranking. Answers with no usable value are rare, under 2% of questions for every model, so the harness returns an answer almost every time.

5.2 Strategy

Figure 3 shows how each model's work grows with the input. From dataset-v1 to dataset-v2, the text per question grows about ten times, from 0.97 MB to 9.3 MB (Table A2). GPT-6.1 Sol and GPT-6 Astra are the outliers: their tokens grow 5.1 and 4.7 times, and they begin to delegate (Table A3). The other OpenAI models grow 2.4 to 2.9 times. Anthropic's Opus and Fable models grow 2.1 to 2.2

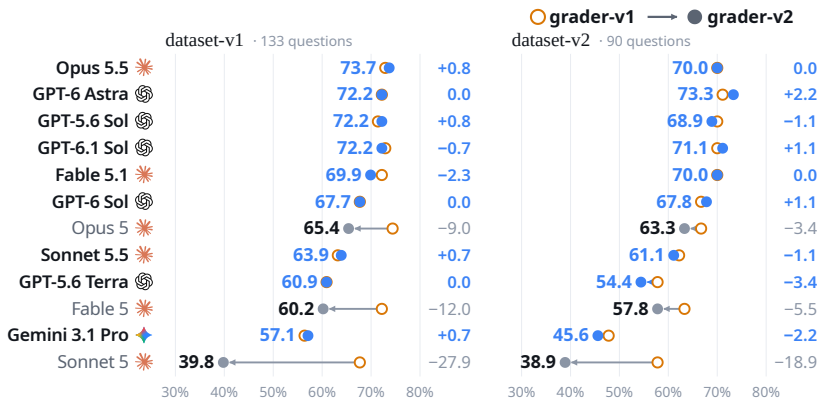


Figure 2: The same trajectories scored by the two graders. Grey rows are the three models that lose the most on dataset-v1: Claude Sonnet 5, Fable 5, and Opus 5.

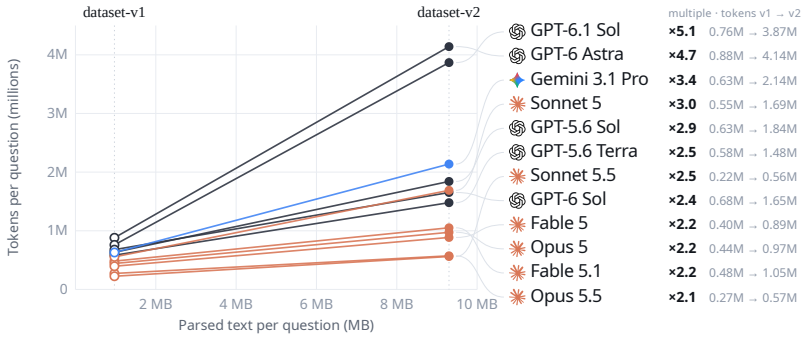


Figure 3: Tokens per question against input size; input sizes from Table A2, tokens from Tables A4 and A5.

times and almost never delegate. As the input grows, the OpenAI models become less token-efficient than Anthropic's Opus and Fable models.

The rest of this section looks at dataset-v2, where the differences are largest.

Every model works in the same harness with the same tools, yet Figure 4 shows that they solve the same questions in very different ways, and that more work buys little accuracy at the top. On dataset-v2, GPT-6 Astra is the most accurate model at 73.3%, with 2.2 subagents, 56 turns, and 143 tool calls per question, taking 15 minutes. Claude Opus 5.5 reaches 70.0% with no subagents, 11 turns, and 13 tool calls, in about a minute. The 3.3 extra points cost Astra about eleven times the tool calls. With 90 questions, each accuracy has a standard error of about 5 points (Section 3.4), so this gap is within noise.

The tools are the same for every model, but how the models use them differs. The GPT-6 models lean on reading files, 59% of GPT-6 Astra's dataset-v2 calls and 50% of GPT-6 Sol's, while GPT-5.6 Sol and Terra split theirs between reading and grep, about 35% each. Anthropic's Opus and Fable models spread their calls across reading, grep, the shell, and task tracking, with no tool above 31%. Gemini 3.1 Pro writes and runs code while barely reading the documents: on dataset-v2, 58% of its calls are bash and 1% are file reads, which likely explains its low accuracy. This pattern comes from the model rather than the harness: the other eleven models read and search the documents with the same tools. None of this appears in a final score; it shows only in the trajectories.

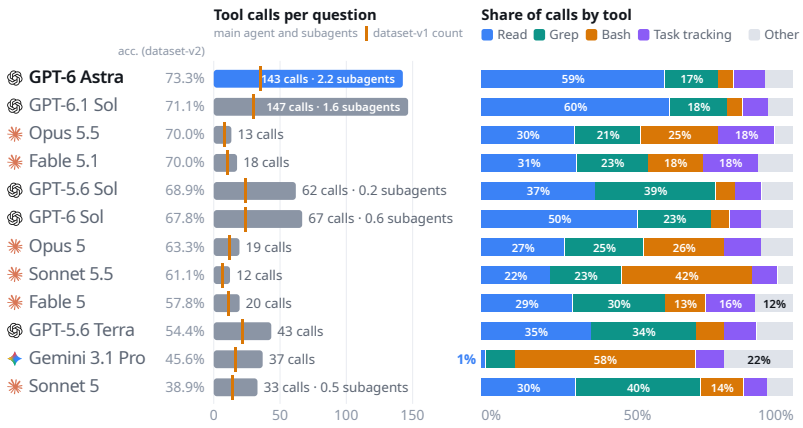


Figure 4: How each model works a dataset-v2 question, sorted by accuracy. Left: tool calls per question, counting the main agent and its subagents, with subagents per question where at least 0.1; the amber tick marks the dataset-v1 count. Right: each model's share of calls by tool; Other is mainly writing files and web search. Table A3 in Appendix C gives every number for both datasets, including turns and time.

5.3 Cost

Figure 1 places each model as a leaderboard does, by one cost per question. That number hides what the cost is made of. By Equation (4), cost is the number of tokens times the effective price per token. On dataset-v2, GPT-6 Astra costs $11.46 per question, 9.2 times Claude Fable 5.1's $1.24, for 3.3 points more accuracy. The 9.2 times is 3.9 times the tokens and 2.3 times the price per token (Tables A5 and A7 in Appendix D). The tokens come from the strategy of Section 5.2, where Astra's subagents read whole documents.



Figure 5: GPT-6 Astra's price per million tokens on dataset-v2, moved to Claude Fable 5.1's one factor at a time: Fable 5.1's cache-read price, then Astra's cache misses served from the cache, then the token mix. Tables A5 and A7 in Appendix D give the full token and cost breakdowns for every model.

Part of Astra's higher price per token comes from its list price. We pair Astra with Fable 5.1 because the two have the same list price for every kind of token except cache reads, which cost Astra four times as much, $1.00 against $0.25 per million (Table A8 in Appendix D). Cache reads are 88.1% of Astra's tokens and 94.8% of Fable 5.1's (Table A5), so this one price carries weight. At Fable 5.1's cache-read price, Astra's price per token would fall from $2.77 to $2.11 per million (Figure 5).

The other factor is the cache. Astra's cache miss rate is 3.2% by Equation (2), the highest among the OpenAI and Anthropic models (Table A5). The misses are specific to Astra: GPT-6 Sol, on the same OpenAI API, misses 0.4%, and Fable 5.1 and Opus 5.5 miss none. Our data cannot tell whether

they come from the model or from how OpenAI serves it. Each missed token is written back into the cache at $12.50 per million instead of read at $1.00. Serving these tokens from the cache as well would lower Astra's price per token further, from $2.11 to $1.72 per million.

Astra then pays the same list prices as Fable 5.1, so the remaining gap, $1.72 against $1.18, comes from its mix of tokens. Of that $0.54 per million, $0.39 is cache writes: even without misses, every call writes its new context once, and Astra's calls add more of it, mostly tool results such as the documents it reads. Another $0.12 is output, reasoning and tool calls included, which is a larger share of Astra's tokens.

6 Conclusion

The leaderboards could not tell us why their top model disappointed our users. Holding every setting fixed except the model and its endpoint, we scored twelve frontier models on both OfficeQA datasets under two grader versions. Three findings surprised us, and each says what to check.

- **Change the grader and you change the winner.** Switching grader version moves accuracy by up to 28 points, mostly because the newer grader rejects right values written in a different form. **Our advice:** score each model under more than one grader to see how much its score moves, and read the answers that flip. If only the form is wrong, fix the grader. If the value is wrong, change the model.
- **The score says how often a model fails, but only the trajectory says why.** GPT-6 Astra and Claude Fable 5.1 score within noise of each other by different routes: Astra hands documents to subagents, 59% reading and 17% searching; Fable 5.1 searches for what it needs, 31% reading and 23% searching. Both fit the task. Gemini 3.1 Pro does not: 1% reading and 9% searching. **Our advice:** read each model's trajectory and ask whether the work fits the task; for questions over documents, that means searching and reading them. If it does not, the model is doing the wrong work, whatever its score.
- **The price sheet is not the bill.** GPT-6 Astra costs 9.2 times Claude Fable 5.1 per question, and the gap has two parts. The tokens are 3.9 times more. The price per token is 2.3 times higher, although the two share list prices for everything but cache reads: cache reads are about 90% of tokens, Astra's cache-read price is four times Fable 5.1's, and Astra also has a 3.2% cache-miss rate. **Our advice:** the bill is the number of tokens times the price per token, so check each part. If the tokens are many, read the trajectory to see where they go. If the price per million is high, look at the list prices and the cache-miss rate.

None of the three shows in a leaderboard score; all three were in the trajectories of the same single run per model that produced each score, and could have told us before our users did. So we ask two things. Teams serving agents: run these checks on your own tasks before you ship, and keep running them after. Leaderboards: state the grader version and setup beside each score, with the standard error, tokens by type, and cache hit rate. OfficeQA already makes this easy: its grader is public, versioned code, so each score can name the version that produced it. A score is only as honest as the work behind it, so we will release ours: every trajectory, so that every number in this paper can be recomputed.

7 Limitations

Our study covers one benchmark and one harness. OfficeQA's tasks are one type of knowledge work, numerical questions over government financial reports; other kinds of knowledge work and other harnesses may favor different models and strategies. For cost reasons, we report one run per model and dataset, so we report no run-to-run variance, whereas Anthropic averages five runs. We give models the oracle documents as parsed text; since OfficeQA scores are sensitive to the harness and lower on raw PDFs (Section 4.1), rankings in the PDF or whole-corpus settings may differ. We measure scaling at only two input sizes, about 1 MB and 10 MB of text per question, on datasets that also differ in corpus; larger inputs are untested. We choose one model per harness; choosing per question from these signals is untested. We cannot tell whether GPT-6 Astra's cache misses come from the model or from how OpenAI serves it; separating the two needs the same model on another endpoint. Costs use list prices as of September 2026; prices change, and because we release token counts by type, costs can be recomputed at any prices.

References

[1] Anthropic. Claude Fable 5.1 and Claude Mythos 5.1 system card. https://www.anthropic.com/claude-fable-5-1-mythos-5-1-system-card, September 2026. Table 8.1.A, p. 167; Section 8.10, p. 176; Section 8.13, p. 187. Section 8.15.1, p. 192. Accessed 23 September 2026.

[2] Scale AI. Humanity's Last Exam leaderboard. https://labs.scale.com/leaderboard/humanitys_last_exam, 2026. Accessed 23 September 2026.

[3] Databricks AI Research. Introducing OfficeQA: A benchmark for end-to-end grounded reasoning. https://www.databricks.com/blog/introducing-officeqa-benchmark-end-to-end-grounded-reasoning, 2025. Blog post, 9 December 2025. Accessed 23 September 2026.

[4] Krista Opsahl-Ong, Arnav Singhvi, Jasmine Collins, Ivan Zhou, Cindy Wang, Ashutosh Baheti, Owen Oertell, Jacob Portes, Sam Havens, Erich Elsen, Michael Bendersky, Matei Zaharia, and Xing Chen. OfficeQA Pro: An enterprise benchmark for end-to-end grounded reasoning. *arXiv preprint arXiv:2603.08655*, 2026.

[5] Shivalika Singh, Yiyang Nan, Alex Wang, Daniel D'Souza, Sayash Kapoor, Ahmet Üstün, Sanmi Koyejo, Yuntian Deng, Shayne Longpre, Noah A. Smith, Beyza Ermis, Marzieh Fadaee, and Sara Hooker. The leaderboard illusion. *arXiv preprint arXiv:2504.20879*, 2025.

[6] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-Judge with MT-Bench and chatbot arena. *arXiv preprint arXiv:2306.05685*, 2023.

[7] Arjun Panickssery, Samuel R. Bowman, and Shi Feng. LLM evaluators recognize and favor their own generations. *arXiv preprint arXiv:2404.13076*, 2024.

[8] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM Test-Time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.

[9] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*, 2025.

[10] Alejandro Cuadron, Dacheng Li, Wenjie Ma, Xingyao Wang, Yichuan Wang, Siyuan Zhuang, Shu Liu, Luis Gaspar Schroeder, Tian Xia, Huanzhi Mao, Nicholas Thumiger, Aditya Desai, Ion Stoica, Ana Klimovic, Graham Neubig, and Joseph E. Gonzalez. The danger of overthinking: Examining the Reasoning-Action dilemma in agentic tasks. *arXiv preprint arXiv:2502.08235*, 2025.

[11] Lovish Madaan, Aaditya K. Singh, Rylan Schaeffer, Andrew Poulton, Sanmi Koyejo, Pontus Stenetorp, Sharan Narang, and Dieuwke Hupkes. Quantifying variance in evaluation benchmarks. *arXiv preprint arXiv:2406.10229*, 2024.

[12] Evan Miller. Adding error bars to evals: A statistical approach to language model evaluations. *arXiv preprint arXiv:2411.00640*, 2024.

[13] Andreas Hochlehnert, Hardik Bhatnagar, Vishaal Udandarao, Samuel Albanie, Ameya Prabhu, and Matthias Bethge. A sober look at progress in language model reasoning: Pitfalls and paths to reproducibility. *arXiv preprint arXiv:2504.07086*, 2025.

[14] Stella Biderman, Hailey Schoelkopf, Lintang Sutawika, Leo Gao, Jonathan Tow, Baber Abbasi, Alham Fikri Aji, Pawan Sasanka Ammanamanchi, Sidney Black, Jordan Clive, Anthony DiPofi, Julen Etxaniz, Benjamin Fattori, Jessica Zosa Forde, Charles Foster, Jeffrey Hsu, Mimansa Jaiswal, Wilson Y. Lee, Haonan Li, Charles Lovering, Niklas Muennighoff, Ellie Pavlick, Jason Phang, Aviya Skowron, Samson Tan, Xiangru Tang, Kevin A. Wang, Genta Indra Winata, François Yvon, and Andy Zou. Lessons from the trenches on reproducible evaluation of language models. *arXiv preprint arXiv:2405.14782*, 2024.

[15] Yunbei Zhang, Janet Wang, Yingqiang Ge, Weijie Xu, Jihun Hamm, and Chandan K. Reddy. Stop comparing LLM agents without disclosing the harness. *arXiv preprint arXiv:2605.23950*, 2026.

[16] V. S. Raghu Parupudi. There is no neutral harness: Modern LLM leaderboards are manufactured by Config-Fragile items. *arXiv preprint arXiv:2608.21382*, 2026.

[17] Naman Vats and Oleg Golev. The scaffold effect in coding agents: Harness choice as a hidden variable in Coding-Agent evaluation. *arXiv preprint arXiv:2607.22585*, 2026.

[18] Moran Mizrahi, Guy Kaplan, Dan Malkin, Rotem Dror, Dafna Shahaf, and Gabriel Stanovsky. State of what art? a call for Multi-Prompt LLM evaluation. *arXiv preprint arXiv:2401.00595*, 2023.

[19] Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models' sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting. *arXiv preprint arXiv:2310.11324*, 2023.

[20] Norah Alzahrani, Hisham Abdullah Alyahya, Yazeed Alnumay, Sultan Alrashed, Shaykhah Alsubaie, Yusef Almushaykeh, Faisal Mirza, Nouf Alotaibi, Nora Altwairesh, Areeb Alowisheq, M Saiful Bari, and Haidar Khan. When benchmarks are targets: Revealing the sensitivity of large language model leaderboards. *arXiv preprint arXiv:2402.01781*, 2024.

[21] Inioluwa Deborah Raji, Emily M. Bender, Amandalynne Paullada, Emily Denton, and Alex Hanna. AI and the everything in the whole wide world benchmark. *arXiv preprint arXiv:2111.15366*, 2021.

[22] Yuxuan Zhu, Tengjun Jin, Yada Pruksachatkun, Andy Zhang, Shu Liu, Sasha Cui, Sayash Kapoor, Shayne Longpre, Kevin Meng, Rebecca Weiss, Fazl Barez, Rahul Gupta, Jwala Dhamala, Jacob Merizian, Mario Giulianelli, Harry Coppock, Cozmin Ududec, Jasjeet Sekhon, Jacob Steinhardt, Antony Kellermann, Sarah Schwettmann, Matei Zaharia, Ion Stoica, Percy Liang, and Daniel Kang. Establishing best practices for building rigorous agentic benchmarks. *arXiv preprint arXiv:2507.02825*, 2025.

[23] Zhi Rui Tam, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung yi Lee, and Yun-Nung Chen. Let me speak freely? a study on the impact of format restrictions on performance of large language models. *arXiv preprint arXiv:2408.02442*, 2024.

[24] Rylan Schaeffer, Brando Miranda, and Sanmi Koyejo. Are emergent abilities of large language models a mirage? *arXiv preprint arXiv:2304.15004*, 2023.

[25] Hwiyeol Jo, Joosung Lee, Jaehone Lee, Sang-Woo Lee, Joonsuk Park, and Kang Min Yoo. Finding answers in thought matters: Revisiting evaluation on large language models with reasoning. *arXiv preprint arXiv:2510.14773*, 2025.

[26] Sayash Kapoor, Benedikt Stroebl, Peter Kirgis, Nitya Nadgir, Zachary S Siegel, Boyi Wei, Tianci Xue, Ziru Chen, Felix Chen, Saiteja Utpala, Franck Ndzomga, Dheeraj Oruganty, Sophie Luskin, Kangheng Liu, Botao Yu, Amit Arora, Dongyoon Hahm, Harsh Trivedi, Huan Sun, Juyong Lee, Tengjun Jin, Yifan Mai, Yifei Zhou, Yuxuan Zhu, Rishi Bommasani, Daniel Kang, Dawn Song, Peter Henderson, Yu Su, Percy Liang, and Arvind Narayanan. Holistic agent leaderboard: The missing infrastructure for AI agent evaluation. *arXiv preprint arXiv:2510.11977*, 2025.

[27] Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. AI agents that matter. *arXiv preprint arXiv:2407.01502*, 2024.

[28] Mehmet Hamza Erol, Batu El, Mirac Suzgun, Mert Yuksekgonul, and James Zou. Cost-of-Pass: An economic framework for evaluating language models. *arXiv preprint arXiv:2504.13359*, 2025.

[29] Databricks. OfficeQA repository: evaluation code, corpus tooling, and result charts. `https://github.com/databricks/officeqa`, 2026. README result charts and the grader reward.py with its commit history. Accessed 24 September 2026.

[30] Databricks AI Research. Introducing OfficeQA Pro V2: A new benchmark for enterprise grounded-reasoning. https://www.databricks.com/blog/introducing-officeqa-pro-v2-new-benchmark-enterprise-grounded-reasoning, 2026. Blog post, 6 August 2026. Accessed 24 September 2026.

[31] Anthropic. Claude Opus 5.5 system card. System card PDF linked from https://www.anthropic.com/claude-opus-5-5, September 2026. Table 8.1.A, p. 174, and Section 8.5, p. 178. Section 8.14.1, p. 208. Accessed 23 September 2026.

[32] Anthropic. Claude Sonnet 5.5 system card. https://www.anthropic.com/claude-sonnet-5-5-system-card, September 2026. Table 8.1.A, p. 109; effort levels, p. 112; Section 8.14.1, p. 132. Accessed 29 September 2026.

[33] Anthropic. Claude Sonnet 5 system card. https://www-cdn.anthropic.com/283ef97c476cf442c91d9a37d5b214242a55bb92/Claude%20Sonnet%205%20System%20Card.pdf, June 2026. 30 June 2026. Section 8.11.1, p. 131. Accessed 24 September 2026.

[34] Anthropic. Claude Opus 5 system card. https://www-cdn.anthropic.com/ceaf5c7ff2783855203fde8208ec311252dced5b/Claude%20Opus%205%20System%20Card.pdf, July 2026. 24 July 2026. Section 8.13.1, p. 180. Accessed 24 September 2026.

[35] E2B. E2B: Open-source, secure environment with real-world tools for enterprise-grade agents. https://github.com/e2b-dev/E2B, 2026. Accessed 29 September 2026.

A Setup

Table A1 lists the twelve models with the endpoint and model ID that served each. Table A2 gives the size of each dataset, counted from the released question files and parsed corpora.

Table A1: The twelve models and the endpoints that served them. All other settings are shared and stated in Section 3.2.

Model	Endpoint	Model ID
Claude Opus 5.5	AWS Bedrock	`global.anthropic.claude-opus-5-5`
Claude Sonnet 5.5	AWS Bedrock	`global.anthropic.claude-sonnet-5-5`
Claude Fable 5.1	Anthropic	`claude-fable-5-1`
Claude Fable 5	Anthropic	`claude-fable-5`
Claude Opus 5	Anthropic	`claude-opus-5`
Claude Sonnet 5	Anthropic	`claude-sonnet-5`
GPT-6 Astra	OpenAI	`gpt-6-astra`
GPT-6 Sol	OpenAI	`gpt-6-sol`
GPT-6.1 Sol	OpenAI	`gpt-6.1-sol`
GPT-5.6 Sol	Azure OpenAI	`gpt-5.6-sol`
GPT-5.6 Terra	Azure OpenAI	`gpt-5.6-terra`
Gemini 3.1 Pro	Google AI Studio	`gemini-3.1-pro-preview`

Table A2: The two datasets in our setting. Counted from the released question files and parsed corpora, at the versions stated in Section 4.1.

	dataset-v1	dataset-v2
Questions	133	90
Documents in corpus	697	1,435
Pages in corpus	88,956	118,524
Pages per document, mean (median, max)	127.6 (126, 280)	82.6 (15, 1,016)
Documents cited	188	249
Documents per question, mean (median, max)	1.8 (1, 5)	6.7 (5.5, 24)
Pages per question, mean (median, max)	237 (180, 901)	2,289 (1,755, 7,909)
Parsed text per question, mean (median, max)	0.97 MB (0.75, 4.3)	9.3 MB (6.2, 36.2)

B Accuracy

Figure A1 splits each model's wrong answers under grader-v2 into four kinds, as summarized in Section 5.1.

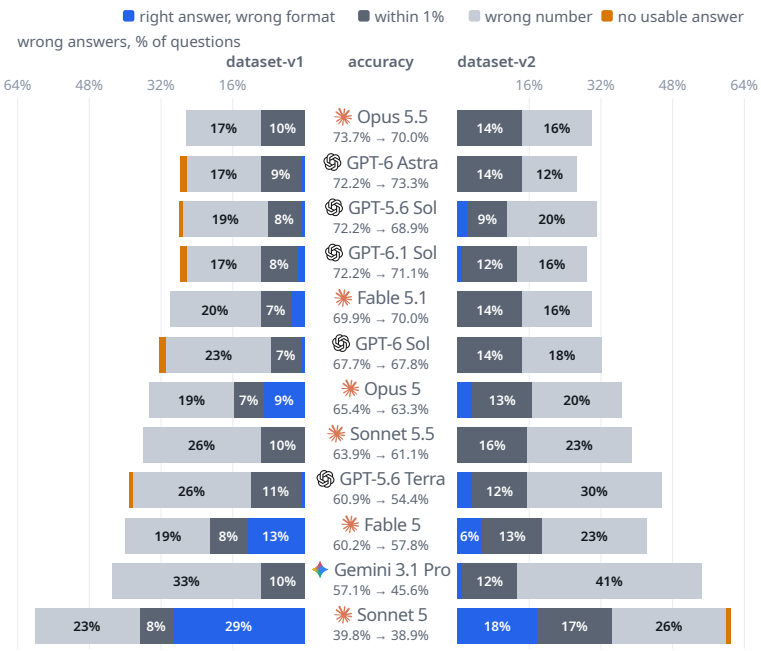


Figure A1: Wrong answers under grader-v2, by kind, as a share of questions; the centre of each row gives the model's accuracy on dataset-v1 and dataset-v2. Right answer, wrong format: the value matches the target but the direct-answer rules reject the answer. Within 1%: off by at most 1%. No usable answer: no value where one was required.

C Strategy

Table A3 gives the numbers behind Figure 4 for both datasets, including turns and time (Section 5.2).

Table A3: How each model works a question, averaged per question and sorted by accuracy. Counts include both the main agent and its subagents. Turns: times the model is called. Tool columns: uses of each tool, with its share of all tool uses in brackets; these four tools account for at least 77% of each model's uses, and the rest are other tools, mainly writing files and web search.

Model	Acc. (%)	Time (min)	Subagents	Turns	Tool calls	Read	Grep	Bash	Task tracking
dataset-v1 (133)									
Claude Opus 5.5	73.7	0.7	0.00	6.7	7.8	1.8 (24%)	1.6 (21%)	1.7 (22%)	2.0 (26%)
GPT-6 Astra	72.2	5.0	0.64	20.1	35.3	13.1 (37%)	4.4 (12%)	3.3 (9%)	6.8 (19%)
GPT-5.6 Sol	72.2	2.3	0.01	13.6	23.9	4.9 (20%)	8.5 (36%)	2.0 (9%)	4.1 (17%)
GPT-6.1 Sol	72.2	3.8	0.40	16.9	29.9	10.9 (36%)	4.2 (14%)	3.0 (10%)	5.7 (19%)
Claude Fable 5.1	69.9	1.7	0.00	11.6	10.7	2.7 (25%)	1.8 (17%)	1.9 (18%)	3.1 (29%)
GPT-6 Sol	67.7	2.2	0.03	14.2	24.3	7.6 (31%)	5.4 (22%)	2.5 (10%)	4.4 (18%)
Claude Opus 5	65.4	1.4	0.00	9.7	11.6	2.9 (25%)	2.9 (25%)	2.4 (20%)	2.4 (21%)
Claude Sonnet 5.5	63.9	0.5	0.00	5.7	6.8	1.7 (24%)	1.8 (26%)	1.6 (24%)	1.0 (14%)
GPT-5.6 Terra	60.9	1.7	0.01	13.7	21.6	3.8 (18%)	8.2 (38%)	2.5 (12%)	4.2 (19%)
Claude Fable 5	60.2	1.6	0.00	8.5	11.0	2.9 (27%)	2.5 (23%)	1.5 (13%)	2.8 (26%)
Gemini 3.1 Pro	57.1	1.3	0.01	17.6	16.1	1.2 (7%)	3.8 (24%)	5.2 (32%)	3.2 (20%)
Claude Sonnet 5	39.8	2.2	0.06	12.3	14.0	3.0 (22%)	5.4 (38%)	2.0 (14%)	2.0 (14%)
dataset-v2 (90)									
GPT-6 Astra	73.3	14.8	2.24	56.3	142.6	83.9 (59%)	24.4 (17%)	6.9 (5%)	14.4 (10%)
GPT-6.1 Sol	71.1	14.5	1.64	51.2	146.6	88.6 (60%)	26.9 (18%)	7.1 (5%)	12.0 (8%)
Claude Opus 5.5	70.0	1.2	0.00	10.7	13.3	4.0 (30%)	2.8 (21%)	3.3 (25%)	2.4 (18%)
Claude Fable 5.1	70.0	3.2	0.01	18.1	17.6	5.4 (31%)	4.0 (23%)	3.1 (18%)	3.1 (18%)
GPT-5.6 Sol	68.9	6.9	0.17	24.3	61.9	22.6 (37%)	23.9 (39%)	3.9 (6%)	5.2 (8%)
GPT-6 Sol	67.8	4.6	0.57	27.6	66.7	33.5 (50%)	15.6 (23%)	4.0 (6%)	6.7 (10%)
Claude Opus 5	63.3	2.7	0.00	17.0	19.4	5.2 (27%)	4.9 (25%)	5.0 (26%)	2.3 (12%)
Claude Sonnet 5.5	61.1	1.1	0.00	10.6	12.3	2.7 (22%)	2.8 (23%)	5.1 (42%)	1.0 (8%)
Claude Fable 5	57.8	4.0	0.04	14.8	19.5	5.7 (29%)	5.8 (30%)	2.5 (13%)	3.1 (16%)
GPT-5.6 Terra	54.4	3.8	0.07	20.7	43.4	15.3 (35%)	14.6 (34%)	3.9 (9%)	4.5 (10%)
Gemini 3.1 Pro	45.6	5.3	0.00	38.4	36.9	0.5 (1%)	3.5 (9%)	21.3 (58%)	3.4 (9%)
Claude Sonnet 5	38.9	7.0	0.50	25.9	33.0	10.0 (30%)	13.2 (40%)	4.5 (14%)	2.5 (8%)

D Cost

Tables A4 and A5 break each model's tokens per question on dataset-v1 and dataset-v2 into the four token types, Tables A6 and A7 do the same for cost, and Table A8 gives the list prices of GPT-6 Astra and Claude Fable 5.1 (Section 5.3).

Table A4: Tokens per question on dataset-v1, averaged over its questions and sorted by accuracy, counting the main agent and its subagents. Each type's share of the row total, in %, is in brackets. Input: tokens sent uncached. A value shown as 0.000 is nonzero but below 0.0005. Hit and Miss: the cache hit rate and cache miss rate, Equations (1) and (2); the remaining $100 - \text{hit} - \text{miss}$ are new tokens. Gemini 3.1 Pro has no cache-write charge; its cache misses are billed as input.

Model (endpoint)	Acc. (%)	Total (M)	Input (M)	Cache read (M)	Cache write (M)	Output (M)	Hit (%)	Miss (%)
Claude Opus 5.5 (AWS Bedrock)	73.7	0.272	0.000 (0.0)	0.255 (93.6)	0.015 (5.4)	0.003 (1.0)	94.6	0.0
GPT-6 Astra (OpenAI)	72.2	0.881	0.000 (0.0)	0.766 (86.9)	0.106 (12.0)	0.009 (1.0)	87.8	3.9
GPT-5.6 Sol (Azure OpenAI)	72.2	0.633	0.001 (0.1)	0.585 (92.4)	0.042 (6.7)	0.006 (0.9)	93.2	0.2
GPT-6.1 Sol (OpenAI)	72.2	0.759	0.000 (0.0)	0.685 (90.2)	0.068 (8.9)	0.007 (0.9)	91.0	0.5
Claude Fable 5.1 (Anthropic)	69.9	0.483	0.000 (0.0)	0.458 (94.9)	0.020 (4.2)	0.004 (0.9)	95.8	0.0
GPT-6 Sol (OpenAI)	67.7	0.678	0.000 (0.0)	0.632 (93.2)	0.041 (6.1)	0.005 (0.8)	93.9	0.0
Claude Opus 5 (Anthropic)	65.4	0.442	0.000 (0.0)	0.413 (93.3)	0.024 (5.5)	0.005 (1.2)	94.5	0.0
Claude Sonnet 5.5 (AWS Bedrock)	63.9	0.225	0.000 (0.0)	0.209 (92.8)	0.014 (6.1)	0.002 (1.0)	93.8	0.0
GPT-5.6 Terra (Azure OpenAI)	60.9	0.583	0.000 (0.1)	0.541 (92.8)	0.036 (6.2)	0.005 (0.9)	93.6	0.2
Claude Fable 5 (Anthropic)	60.2	0.395	0.000 (0.0)	0.366 (92.6)	0.024 (6.1)	0.005 (1.4)	93.8	0.0
Gemini 3.1 Pro (Google AI Studio)	57.1	0.630	0.067 (10.7)	0.558 (88.6)	0.000 (0.0)	0.004 (0.7)	89.3	0.0
Claude Sonnet 5 (Anthropic)	39.8	0.555	0.000 (0.0)	0.514 (92.6)	0.031 (5.6)	0.010 (1.8)	94.3	0.4

Table A5: Tokens per question on dataset-v2, averaged over the 90 questions and sorted by accuracy, counting the main agent and its subagents. Each type's share of the row total, in %, is in brackets. Input: tokens sent uncached. A value shown as 0.000 is nonzero but below 0.0005. Hit and Miss: the cache hit rate and cache miss rate, Equations (1) and (2); the remaining $100 - \text{hit} - \text{miss}$ are new tokens. Gemini 3.1 Pro has no cache-write charge; its cache misses are billed as input.

Model (endpoint)	Acc. (%)	Total (M)	Input (M)	Cache read (M)	Cache write (M)	Output (M)	Hit (%)	Miss (%)
GPT-6 Astra (OpenAI)	73.3	4.141	0.007 (0.2)	3.649 (88.1)	0.440 (10.6)	0.045 (1.1)	89.1	3.2
GPT-6.1 Sol (OpenAI)	71.1	3.868	0.010 (0.3)	3.452 (89.3)	0.368 (9.5)	0.038 (1.0)	90.1	1.2
Claude Opus 5.5 (AWS Bedrock)	70.0	0.573	0.000 (0.0)	0.534 (93.2)	0.033 (5.8)	0.006 (1.0)	94.1	0.0
Claude Fable 5.1 (Anthropic)	70.0	1.051	0.000 (0.0)	0.996 (94.8)	0.046 (4.3)	0.009 (0.8)	95.6	0.0
GPT-5.6 Sol (Azure OpenAI)	68.9	1.841	0.002 (0.1)	1.695 (92.1)	0.129 (7.0)	0.015 (0.8)	92.8	0.7
GPT-6 Sol (OpenAI)	67.8	1.653	0.000 (0.0)	1.529 (92.5)	0.109 (6.6)	0.015 (0.9)	93.4	0.4
Claude Opus 5 (Anthropic)	63.3	0.973	0.000 (0.0)	0.921 (94.6)	0.042 (4.3)	0.010 (1.1)	95.7	0.0
Claude Sonnet 5.5 (AWS Bedrock)	61.1	0.561	0.000 (0.0)	0.521 (92.8)	0.035 (6.2)	0.006 (1.0)	93.7	0.0
Claude Fable 5 (Anthropic)	57.8	0.885	0.000 (0.0)	0.822 (92.8)	0.050 (5.7)	0.013 (1.5)	94.2	0.3
GPT-5.6 Terra (Azure OpenAI)	54.4	1.478	0.000 (0.0)	1.380 (93.3)	0.087 (5.9)	0.011 (0.8)	94.0	0.4
Gemini 3.1 Pro (Google AI Studio)	45.6	2.138	0.209 (9.8)	1.911 (89.4)	0.000 (0.0)	0.018 (0.8)	90.1	6.6
Claude Sonnet 5 (Anthropic)	38.9	1.689	0.000 (0.0)	1.565 (92.6)	0.097 (5.7)	0.027 (1.6)	94.2	0.7

Table A6: Cost per question on dataset-v1 at each model's list price, for the same questions and in the same order as Table A4. Each type's share of the row's cost is in brackets. Price: the effective price per token, total cost divided by total tokens (Section 3.4).

Model (endpoint)	Acc. (%)	Total ($)	Input ($)	Cache read ($)	Cache write ($)	Output ($)	Price ($/M tokens)
Claude Opus 5.5 (AWS Bedrock)	73.7	0.178	0.000 (0%)	0.051 (29%)	0.073 (41%)	0.054 (30%)	0.65
GPT-6 Astra (OpenAI)	72.2	2.531	0.004 (0%)	0.766 (30%)	1.323 (52%)	0.438 (17%)	2.87
GPT-5.6 Sol (Azure OpenAI)	72.2	0.726	0.003 (0%)	0.292 (40%)	0.264 (36%)	0.167 (23%)	1.15
GPT-6.1 Sol (OpenAI)	72.2	0.303	0.001 (0%)	0.068 (23%)	0.169 (56%)	0.065 (22%)	0.40
Claude Fable 5.1 (Anthropic)	69.9	0.581	0.000 (0%)	0.115 (20%)	0.251 (43%)	0.215 (37%)	1.20
GPT-6 Sol (OpenAI)	67.7	0.282	0.000 (0%)	0.126 (45%)	0.103 (36%)	0.052 (19%)	0.42
Claude Opus 5 (Anthropic)	65.4	0.491	0.000 (0%)	0.206 (42%)	0.151 (31%)	0.134 (27%)	1.11
Claude Sonnet 5.5 (AWS Bedrock)	63.9	0.099	0.000 (0%)	0.042 (42%)	0.035 (35%)	0.023 (23%)	0.44
GPT-5.6 Terra (Azure OpenAI)	60.9	0.264	0.001 (0%)	0.108 (41%)	0.091 (34%)	0.064 (24%)	0.45
Claude Fable 5 (Anthropic)	60.2	0.937	0.000 (0%)	0.366 (39%)	0.300 (32%)	0.270 (29%)	2.37
Gemini 3.1 Pro (Google AI Studio)	57.1	0.297	0.134 (45%)	0.112 (38%)	0.000 (0%)	0.051 (17%)	0.47
Claude Sonnet 5 (Anthropic)	39.8	0.280	0.000 (0%)	0.103 (37%)	0.078 (28%)	0.099 (35%)	0.50

Table A7: Cost per question on dataset-v2 at each model's list price, for the same questions and in the same order as Table A5. Each type's share of the row's cost is in brackets. Price: the effective price per token, total cost divided by total tokens (Section 3.4).

Model (endpoint)	Acc. (%)	Total ($)	Input ($)	Cache read ($)	Cache write ($)	Output ($)	Price ($/M tokens)
GPT-6 Astra (OpenAI)	73.3	11.464	0.066 (1%)	3.649 (32%)	5.498 (48%)	2.251 (20%)	2.77
GPT-6.1 Sol (OpenAI)	71.1	1.660	0.020 (1%)	0.345 (21%)	0.920 (55%)	0.375 (23%)	0.43
Claude Opus 5.5 (AWS Bedrock)	70.0	0.386	0.000 (0%)	0.107 (28%)	0.167 (43%)	0.112 (29%)	0.67
Claude Fable 5.1 (Anthropic)	70.0	1.244	0.000 (0%)	0.249 (20%)	0.559 (45%)	0.436 (35%)	1.18
GPT-5.6 Sol (Azure OpenAI)	68.9	2.119	0.008 (0%)	0.848 (40%)	0.807 (38%)	0.457 (22%)	1.15
GPT-6 Sol (OpenAI)	67.8	0.724	0.000 (0%)	0.306 (42%)	0.272 (38%)	0.146 (20%)	0.44
Claude Opus 5 (Anthropic)	63.3	0.983	0.000 (0%)	0.461 (47%)	0.261 (27%)	0.261 (27%)	1.01
Claude Sonnet 5.5 (AWS Bedrock)	61.1	0.247	0.000 (0%)	0.104 (42%)	0.087 (35%)	0.055 (22%)	0.44
Claude Fable 5 (Anthropic)	57.8	1.998	0.000 (0%)	0.798 (40%)	0.594 (30%)	0.606 (30%)	2.25
GPT-5.6 Terra (Azure OpenAI)	54.4	0.631	0.001 (0%)	0.276 (44%)	0.219 (35%)	0.136 (22%)	0.43
Gemini 3.1 Pro (Google AI Studio)	45.6	1.013	0.418 (41%)	0.382 (38%)	0.000 (0%)	0.213 (21%)	0.47
Claude Sonnet 5 (Anthropic)	38.9	0.827	0.001 (0%)	0.313 (38%)	0.242 (29%)	0.271 (33%)	0.49

Table A8: List prices of GPT-6 Astra and Claude Fable 5.1, in dollars per million tokens.

Token type	GPT-6 Astra (OpenAI)	Claude Fable 5.1 (Anthropic)
Input	10.00	10.00
Cache read	1.00	0.25
Cache write	12.50	12.50
Output	50.00	50.00